

The Linux Shell and Filesystem

Understanding the **Linux filesystem**, **file management** fundamentals, and the basics of the **Linux shell** and **command-line interface (CLI)** is essential for a modern-day Linux professional.

In this chapter, you will learn how to use the Linux shell and some of the most common commands in Linux. You will learn about the structure of a basic Linux command and how the Linux filesystem is organized. We'll explore various commands for working with files and directories. Along the way, we'll introduce you to the most common command-line text editors. We hope that by the end of this chapter, you'll be comfortable using the Linux CLI and be ready for future, more advanced explorations. This chapter will set the foundation for using the Linux shell, and for more information about the shell, go to [Chapter 8, Linux Shell Scripting](#).

We're going to cover the following main topics:

- Introducing the Linux shell
- The Linux filesystem
- Working with files and directories
- Using text editors to create and edit files

Technical requirements

This chapter requires a working installation of a standard Linux distribution, on either server, desktop, PC, or **Virtual Machine (VM)**. Our examples and case studies use the Ubuntu and Fedora platforms, but the commands and examples explored are equally suitable for any other Linux distribution.

Introducing the Linux shell

Linux has its roots in the Unix operating system, and one of its main strengths is the command-line interface. In the old days, this was called *the shell*. In **UNIX**, the shell is invoked with the **sh** command. The shell is a program that has two streams: an *input stream* and an *output stream*. The input is a command given by the user, and the output is the result of that command, or an interpretation of it. In other words, the shell is the primary interface between the user and the machine.

The main shell in major Linux distributions is called **Bash**, which is an acronym for **Bourne Again Shell**, named after Steve Bourne, the original creator of the shell in UNIX. Alongside Bash, there are other shells available in Linux, such as **ksh**, **tcsh**, and **zsh**. In this chapter and throughout the book, we will cover the Bash shell, as it is the most widely used shell in modern Linux distributions.

IMPORTANT NOTE

Distributions such as Debian, Ubuntu, Fedora, CentOS Stream, RHEL, openSUSE, SLE, and Linux Mint, just to name a few, use the Bash shell by default. Other distributions, such as Kali Linux, have switched to zsh by default. Manjaro offers zsh on some editions. For those who use macOS, you should know that zsh has been the default shell for some years now. Nevertheless, you can install any shell you want on Linux and make it your default one. In general, shells are pretty similar, as they do the same thing, but they add different extras to usability and features. If you are interested in a specific shell, feel free to use it and test out the differences between others.

One shell can be assigned to each user. Users on the same system can use different shells. One way to check the default shell is by accessing the `/etc/passwd` file. More details about this file and user accounts will be discussed in [Chapter 4, Managing Users and Groups](#). For now, it is important to know where to look for the default shell. In this file, the last characters from each line represent the user's default shell. The `/etc/passwd` file has the users listed on each line, with details about their **process identification number (PID)**, **group identification number (GID)**, username, home directory, and basic shell.

To see the default shell for each user, execute the following command by using your user's name (in our case, it is **packt**):

```
cat /etc/passwd | grep packt
```

The output should be a list of the contents of the `/etc/passwd` file. Depending on the number of users you have on your system, you will see all of them, each one on a separate line. An easier way to see the *current shell* is by running the following command:

```
echo $0
```

This shows what exactly is running your command, which in the case of the CLI is the shell. The `$0` part is a **bash special parameter** that refers to the currently running process. In the following screenshot you will see the output of the previous two commands we used to discover the shell and a comparison between the output of the `echo $0` command on Ubuntu and on Debian:

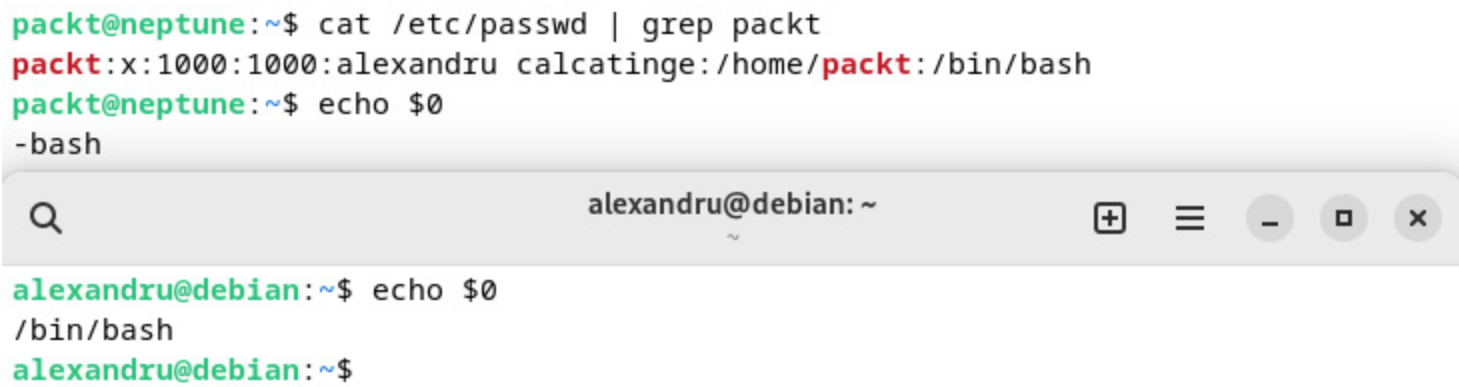


Figure 2.1 – Commands used to see the running shell

As you can see, running the `echo $0` command gives us different outputs but with the same message: the running shell is Bash. If you have other shells that you prefer, you can easily assign another shell to your user, if you already have it installed. However, if you know Bash, you will be comfortable with all the other available shells.

IMPORTANT NOTE

The Linux shell is case-sensitive. This means that everything you type inside the command line should respect this. For example, the `cat` command used earlier used lowercase. If you type `Cat` or `CAT`, the shell will not recognize it as being a command. The same rule applies to file paths. You will notice that default directories in your home directory use uppercase for the first letter, as in `~/Documents`, `~/Downloads`, and so on. Those names are different from `~/documents` or `~/downloads`.

In this chapter, you will learn how to use Linux commands and the shell in addition to learning about its filesystem. You will learn about software management in [Chapter 3](#), thus showing you how to install another shell now would mean that we will get ahead of ourselves. We want you to slowly but steadily build your Linux knowledge, so we will show you in the next chapter how to install a new shell. For now, Bash is sufficient, and we will use it throughout the book.

If you want to see all the shells that are installed on your system, you can run the following command:

```
cat /etc/shells
```

In our case, the output with all the installed shells (by default) in Ubuntu Server 22.04.2 LTS is shown in the following image:

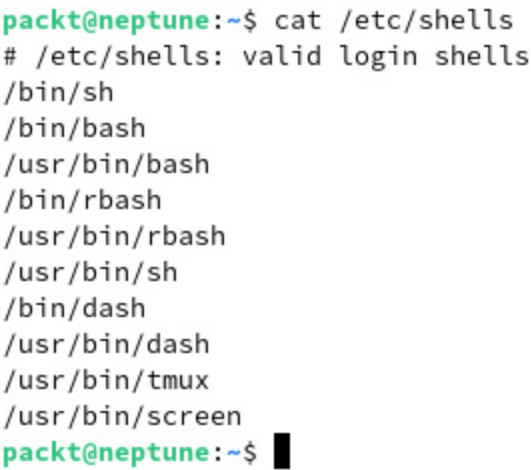


Figure 2.2 – The shells available by default in Ubuntu

This will show you all the shells installed. You can use any of those or can install new ones as we will show you in [Chapter 3](#). Also, in [Chapter 4](#), when we work with user accounts, you will get to learn how you can change a user’s shell.

The following section will introduce you to shell connection types.

Establishing the shell connection

We can make two different types of connections to the shell: `tty` and `pts`. The name `tty` stands for **teletypewriter**, which was a type of terminal used at the beginning of computing. This connection is considered a native one, with ports that are direct connections to your computer. The link between the user and the computer is mainly found to be through a keyboard, which is considered to be a native terminal device.

The `pts` connection is generated by SSH or Telnet types of links. Its name stands for **pseudo terminal slave**, and it is an emulated connection made by a program, in most cases `ssh` or `xterm`. It is the slave of the **pseudo-terminal device**,

which is represented as **pty**.

In the next section, we will further explore the connections to virtual terminals available in Linux.

Virtual consoles/terminals

The terminal was thought of as a device that manages the input strings (which are commands) between a process and other I/O devices such as a keyboard and a screen. There are also **pseudo terminals**, which are emulated terminals that behave the same way as a **classical terminal**. The difference is that it does not interact with devices directly, as it is all emulated by the Linux kernel, which transmits the I/O to a program called the shell.

Virtual consoles are accessible and run in the background, even though there is no open terminal. To access those virtual consoles, you can use the commands *Ctrl + Alt + F1*, *Ctrl + Alt + F2*, *Ctrl + Alt + F3* , *Ctrl + Alt + F4* , *Ctrl + Alt + F5*, and *Ctrl + Alt + F6*. These will open **tty1**, **tty2**, **tty3**, **tty4**, **tty5**, and **tty6**, respectively, on your computer.

We will explain this using an Ubuntu 22.04.2 LTS Server VM installation, but it is identical in Rocky Linux too. After starting the VM and being prompted to log in with your username and password, the first line on the screen will be something similar to the following output:

```
Ubuntu 22.04.2 LTS neptune tty1
```

If you press any of the preceding key combinations, you will see your terminal change from **tty1** to any of the other **tty** instances. For example, if you press *Ctrl + Alt + F6*, you will see this:

```
Ubuntu 22.04.2 LTS neptune tty6
```

As we were using the server edition of Ubuntu, we did not have the GUI installed. But if you were to use a desktop edition, you will be able to use *Ctrl + Alt + F7* to enter **x graphical** mode, for example. The **neptune** string is the name we gave to our virtual machine.

If you are not able to use the preceding keyboard combinations, there is a dedicated command for changing virtual terminals. The command is called **chvt** and has the syntax **chvt N**. Even though we have not discussed shell commands yet, we will show you an example of how to use them and other related commands. This action can only be performed by an administrator account or by using **sudo**. Briefly, **sudo** stands for *superuser do* and allows any user to run programs with administrative privileges or with the privileges of another user (more details about this in [Chapter 4, Managing Users and Groups](#)).

In the following example, we will use Ubuntu to show you how to change virtual terminals. First, we will see which virtual terminal we are currently using to change it to another one without using the *Ctrl + Alt + Fn* keys.

The **who** command will show you information about the users currently logged in to the computer. In our case, as we are connected through SSH to our virtual machine, it will show that the user **packt** is currently using pseudo-terminal zero (**pts/0**):

```
packt pts/0 2023-02-28 10:45 (192.168.122.1)
```

If we were to run the same command in the console of the virtual machine directly, we would have the following output:

```
packt pts/0 2023-02-28 10:45 (192.168.122.1)
packt tty1 2023-02-28 10:50
```

It shows that the user is connected to both the virtual terminal 1 (**tty1**) and also through SSH from our host operating system to the virtual machine (**pts/0**).

Now, by using the **chvt** command, we will show you how to change to the sixth virtual terminal. After running **sudo chvt 6**, you will be prompted to provide your password and immediately be switched to virtual terminal number six. Running **who** once more will show you all logged-in users and the virtual terminals they use. In our case will be **pts/0**, **tty2**, and **tty6**. Please take into consideration that your output could be different, as in different virtual terminal numbers.

Now that we know what types of shell connections are established, let us learn about the shell’s prompt in the next section.

The command-line prompt

The **command-line prompt** or **shell prompt** is the place where you type in the commands. Usually, the command prompt will show the username, hostname, present working directory, and a symbol that indicates the type of user running the shell.

Here is an example from the Ubuntu 22.04.2 LTS Server edition (similar to Debian):

```
packt@saturn:~$
```

Here is an example from the Fedora 37 server (similar to Rocky Linux, RHEL, or AlmaLinux):

```
[packt@localhost ~]$
```

Here is a short explanation of the prompt:

- **packt** is the name of the user currently logged in
- **saturn** and **localhost** are the hostnames
- **~** represents the home directory (it is called a tilde)
- **\$** shows that the user is a regular user (when you are logged in as an administrator, the sign changes into a hashtag, #)

Also, when using openSUSE, you will notice that the prompt is different than the ones in Ubuntu/Debian and Fedora/RHEL. The following is an example of the prompt while running the Leap 15.4 server edition:

```
packt@localhost:~>
```

As you can see, there is no dollar sign (\$) or hashtag (#), only a greater than sign (>). This might be confusing at first, but when you will use the root user, the sign will eventually change to the hashtag (#). The following is an example:

```
localhost:/home/packt #
```

Let’s look at the shell command types next.

Shell command types

Shells work with **commands**, and there are two types that they use: internal ones and external ones. **Internal commands** are built inside the shell. **External commands** are installed separately. If you want to check the type of command you are using, there is the **type** command. For example, you can check what type of command **cd** (change directory) is:

```
packt@neptune:~$ type cd
cd is a shell builtin
```

The output shows that the **cd** command is an internal one, built inside the shell. If you are curious, you could find out the types of other commands that we will show you in the following sections by writing **type** in front of the command’s name. Let us see some more examples in the following image:

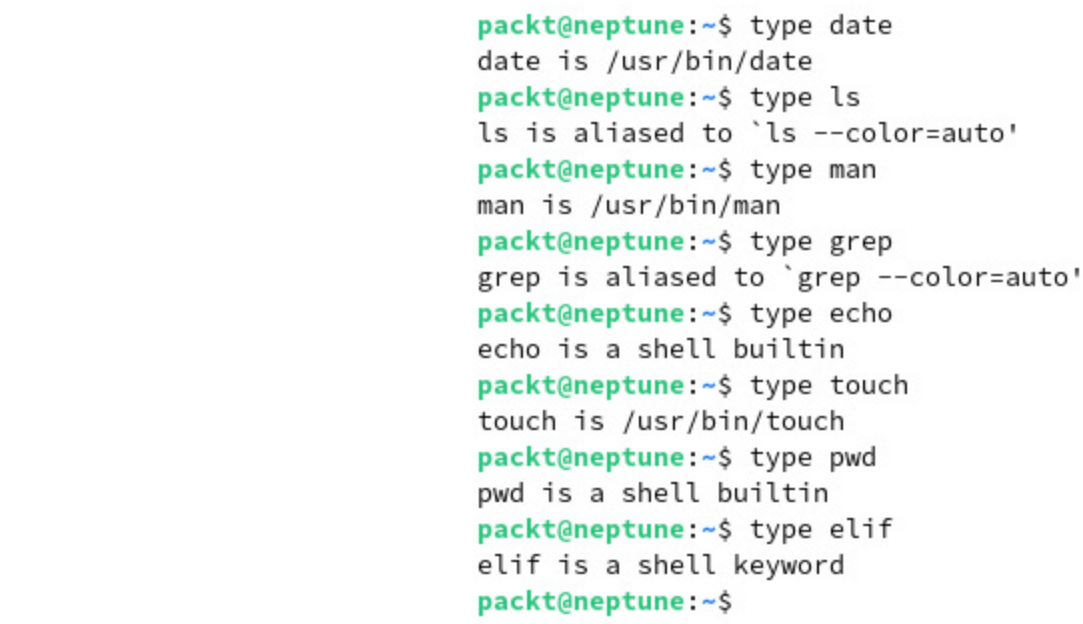


Figure 2.3 – Different types of commands in Linux

Now that you know some of the types of Linux commands, let us dissect the command’s structure and learn about its components.

Explaining the command structure

We have already used some commands, but we did not explain the structure of a Linux command. We will do that now for you to be able to understand how to use commands. In a nutshell, Unix and Linux commands have the following form:

- The command’s name

- The command’s options
- The command’s arguments

Inside the shell, you will have a general structure such as the following:

`command [-option(s)] [argument(s)]`

A suitable example would be the use of the `ls` command (`ls` comes from a *list*). This command is one of the most-used commands in Linux. It lists files and directories and can be used with both options and arguments.

We can use `ls` in its simplest form, without options or arguments. It lists the contents of your present working directory (`pwd`). In our case, it is the home directory, indicated by the `~` tilde character in the shell’s prompt (see *Figure 2.10*).

The `ls` command with the `-l` option (lowercase L) uses a long listing format, giving you extra information about files and directories from your present working directory (`pwd`):

```
packt@saturn:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
packt@saturn:~$ ls -l
total 36
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Desktop
drwxr-xr-x 2 packt packt 4096 mar  1 21:05 Documents
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Downloads
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Music
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Pictures
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Public
drwx----- 3 packt packt 4096 feb 28 12:32 snap
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Templates
drwxr-xr-x 2 packt packt 4096 feb 28 12:32 Videos
packt@saturn:~$ ls -l ~/Documents
total 0
-rw-rw-r-- 1 packt packt 0 mar  1 21:04 file1
-rw-rw-r-- 1 packt packt 0 mar  1 21:04 file2
-rw-rw-r-- 1 packt packt 0 mar  1 21:04 file3
-rw-rw-r-- 1 packt packt 0 mar  1 21:05 report1
-rw-rw-r-- 1 packt packt 0 mar  1 21:05 report2
-rw-rw-r-- 1 packt packt 0 mar  1 21:05 report3
-rw-rw-r-- 1 packt packt 0 mar  1 21:05 report4
-rw-rw-r-- 1 packt packt 0 mar  1 21:05 report5
```

Figure 2.4 – Using the `ls` command with both options and attributes

In the preceding example, we used `ls -l ~/Documents/` to show the contents of the `~/Documents` directory. Shown here is a way to use the command with both options and attributes, without changing our present working directory to `~/Documents`.

In the following section, we will show you how to use the manual pages available by default in Linux.

Consulting the manual

Any Linux system administrator’s best friend is the manual. Each command in Linux has a manual page that gives the user detailed information about its use, options, and attributes. If you know the command you want to learn more about, simply use the `man` command to explore. For the `ls` command, for example, you use `man ls`.

The manual organizes its command information into different sections, with each section being named by convention to be the same on all distributions. Briefly, those sections are **name**, **synopsis**, **configuration**, **description**, **options**, **exit status**, **return value**, **errors**, **environment**, **files**, **versions**, **conforming to**, **notes**, **bugs**, **example**, **authors**, **copyright**, and **see also**.

Similar to the manual pages, almost all commands in Linux have a `-help` option. You can use this for quick reference.

For more information about the `help` and `man` pages, you can check each command’s help or manual page. Try the following commands:

```
$ man man
```